

3 基于大语言模型的漏洞检测增强原理与架构

3.1 研究的问题描述

在小型深度学习漏洞检测模型的实际工程部署中，尽管基于预训练语言模型与神经网络的检测范式已取得显著进展，但面对复杂工业代码场景与严格资源约束时，现有方法仍暴露出明显的局限性。结合当前主流漏洞数据集特性与小模型部署约束，现有研究主要面临以下四个核心瓶颈：

- (1) **上下文窗口受限与长序列语义丢失：**工业界广泛采用的小型预训练模型通常受限于较短的上下文窗口（如 512–1024 tokens），而真实漏洞函数往往跨越数百行，包含大量跨基本块的控制流与数据流依赖。强制截断或简单滑动窗口分块处理会破坏函数级的语义连贯性，导致关键污点传播路径、边界检查缺失及异常处理逻辑断裂，模型难以捕捉隐蔽的跨语句漏洞模式。
- (2) **正负样本极端失衡与梯度淹没：**高质量基准数据集中安全代码占据绝对主导，正负样本比例差距悬殊。在小批量训练条件下，参数量受限的小模型极易被大量易分负样本主导梯度更新方向，产生“梯度淹没”效应。模型在优化过程中倾向于将所有输入预测为安全类以最小化全局损失，导致对少数类漏洞样本的判别边界模糊，召回率长期处于低位。
- (3) **静态增强策略与模型实际表现脱节：**传统数据增强多采用预设规则等价变换、随机噪声注入或基于表面相似度的检索扩增等静态模式，缺乏对模型验证阶段实际薄弱环节的针对性考量。增强样本的分布与模型当前的认知盲区严重错配，不仅易引入低质冗余数据加剧训练集的分布偏移，还会造成大模型调用算力资源的无效消耗，制约检测性能的持续演进。
- (4) **小批量微调易过拟合与输出过度自信：**单一交叉熵损失函数在极端类别失衡分布下难以稳定优化轨迹。模型在训练后期易对训练集噪声与特定代码风格过度记忆，输出概率分布趋于极端化（即预测值逼近 0 或 1）。这种过度自信现象不仅削弱了模型对未见代码的泛化能力，也给实际安全扫描流水线中的风险阈值划定与误报控制带来困难。

针对上述工程约束与算法瓶颈，本文聚焦小型漏洞检测模型的特征提纯、训练优化与数据构建，归纳出本研究需攻克的核心关键问题：

- **关键问题 1：**如何在严格上下文长度约束下，完整保留漏洞触发的核心因果链条，避免因粗暴截断导致的语义丢失？
- **关键问题 2：**如何突破静态盲目扩增的局限，构建以模型实际检测缺陷为驱动的定向难例生成与回流机制？
- **关键问题 3：**如何设计自适应的损失调度策略，在缓解类别失衡的同时抑制小模型过拟合与置信度极端化？

3.2 研究的设计思路

本文研究的核心目标是在有限算力与上下文约束下，实现小型漏洞检测模型判别精度与泛化鲁棒性的自适应提升。针对 3.1 节归纳的关键问题，本研究摒弃传统“一次性数据准备 + 固定损失训练”的单向流水线，提出构建“验证反馈驱动的闭环动态增强框架”。整体设计思路如图3.1所示，遵循以下核心逻辑主线：

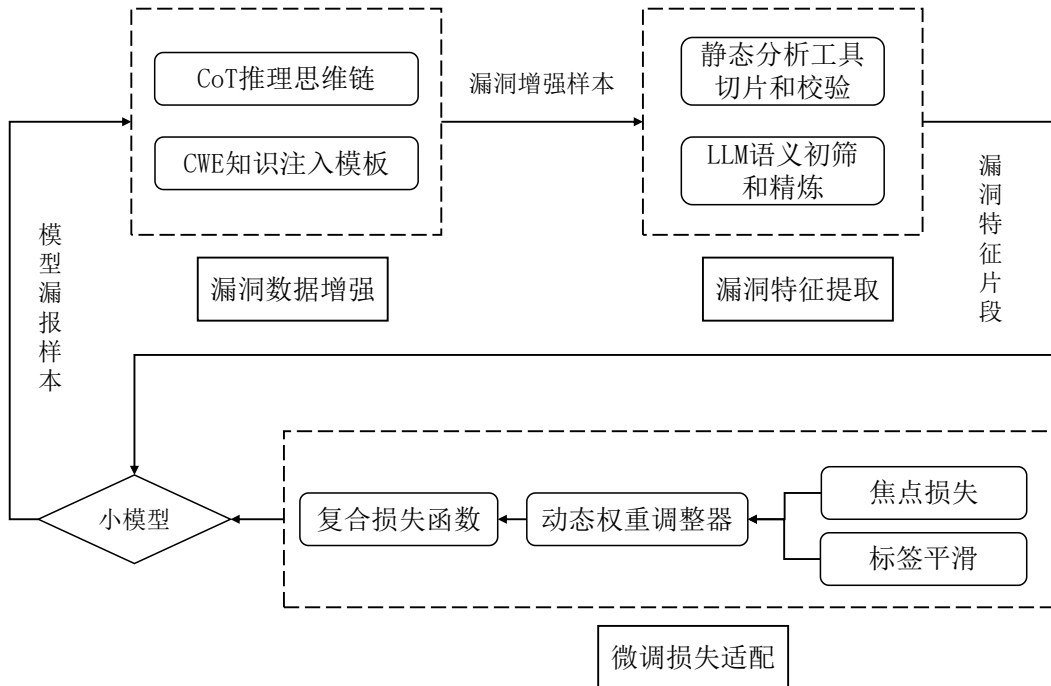


图 3.1 基于大语言模型的漏洞检测增强设计思路

- (1) **以验证难例为动态触发信号：**改变传统增强策略脱离模型训练状态的缺陷，将模型在验证集上暴露的假阴性（FN）漏报样本作为核心反馈信号。通过实时锚

定模型的认知盲区与薄弱特征空间，确保每一轮增强数据的生成方向精准对齐模型当前最迫切的优化需求，实现“数据供给”与“模型需求”的动态匹配。

- (2) **双路协同的结构化特征提纯**：针对长函数输入受限问题，设计“静态分析定位与大语言模型初筛双路并行一大模型融合精炼”机制。依托代码属性图（CPG）提供可严格验证的数据/控制依赖骨架，有效抑制大语言模型的语义幻觉；同时借助大模型的跨上下文推理能力，捕捉静态规则难以形式化的隐式类型转换与业务逻辑缺陷。二者在严格 Token 预算约束下协同输出高信噪比、因果链完整的标准化代码片段。
- (3) **知识约束与逻辑推演并行的定向生成**：针对 FN 种子设计异构双模板注入架构。一方面通过四步递进式思维链（CoT）模拟人类安全审计员的“场景理解—类型对齐—模式抽象—代码重构”推理过程；另一方面显式注入 CWE 官方知识库中的标准化漏洞描述与典型攻击模式，进行领域先验强约束。生成结果统一送入 Tree-sitter 解析器执行 AST 语法校验，构建“生成—校验—有限重试”的质量管控闭环，保障增强代码的结构合法性与漏洞特征纯度。
- (4) **时空解耦的动态损失权重调度**：针对小批量训练下的梯度偏置问题，构建融合焦点损失函数与标签平滑策略的复合目标函数。引入基于训练演进的动态权重调度器，依据迭代轮次自适应调节两项损失的贡献比例，实现优化重心从“初期难例聚焦挖掘”向“后期概率校准维稳”的平滑迁移。

基于上述设计思路，本研究框架具备以下方法论优势：其一，**反馈闭环化**，打破静态增强与动态训练的壁垒，使数据分布随模型能力演进持续优化；其二，**表征结构化**，通过 CPG 与 LLM 的互补融合，在压缩序列长度的同时保留漏洞判定的核心结构先验；其三，**生成可控化**，CWE 知识锚点与语法校验双保险，有效规避大模型自由生成带来的类型漂移与结构破碎；其四，**优化自适应化**，动态损失调度机制使模型在类别极度失衡场景下仍能保持稳定的收敛轨迹与校准后的输出置信度。

3.3 研究的解决方案

针对检测过程中存在的长序列截断语义丢失、静态增强分布错配及小批量微调易过拟合等问题，本文根据 3.2 节的设计思路提出以下三项具体解决方案，以系统性应

对前述关键问题：

解决方案一：基于静态分析与大模型协同的漏洞特征提取方案。针对原始函数长度远超小模型上下文窗口限制、直接截断导致关键语义丢失的问题，构建 Joern 与 LLM 协同的递进式切片流程。首先，利用 Joern 解析原始代码构建代码属性图（CPG），融合 AST、CFG 与 PDG 信息，通过深度优先搜索（DFS）执行双向切片：后向切片精准追溯变量污点传播源头与外部输入边界，前向切片追踪危险 API 调用与危害传播终点，提取具有严格数据/控制依赖的候选片段。随后，将候选切片与 LLM 独立初筛结果共同输入大模型进行二次语义验证与融合对齐。该阶段大模型以静态切片为结构参考基准，智能剔除冗余声明、无关业务逻辑及排版噪声，自动补全跨作用域的关键上下文。该方案在满足小模型输入长度约束的前提下，完整保留了漏洞触发的核心因果链条，为后续参数适配提供标准化、高信噪比的输入基元。

解决方案二：基于验证反馈与异构模板的定向数据增强方案。针对样本极端失衡与静态增强脱节问题，提出“难例捕获一定向生成一语法净化”的闭环机制。系统自动捕获验证阶段的 FN 样本作为增强种子，并行加载 CoT 提示模板与 CWE 定制化知识库模板。CoT 模板引导大模型依次完成“漏洞成因分析—类型识别—模式提取—相似函数生成”的结构化推理，实现从具体实例到抽象规则的逻辑跃迁；CWE 模板则注入官方标准化描述、典型攻击路径与修复范式，约束生成语义空间，避免类型漂移。生成结果统一送入 Tree-sitter 解析器进行 AST 构建与语法校验，严格检测括号匹配、语句终止符及基础作用域闭合。仅将通过校验的高保真样本回流至训练集，失败则触发有限重试。该方案从根本上阻断了无效样本对梯度更新的干扰，定向扩充了高价值正样本，重塑正负样本分布比例。

解决方案三：基于训练演进的动态加权复合损失适配方案。针对小批量微调易过拟合与输出过度自信问题，构建动态加权复合损失函数 $L_{total} = \alpha_t \cdot L_{FL} + \beta_t \cdot L_{LS}$ 。引入动态权重调度器，在微调初期赋予 α_t 较高权重，强化 Focal Loss 调制因子 $(1 - p_t)^\gamma$ 对易分负样本的压制作用，迫使模型梯度更新优先聚焦高漏报的脆弱特征空间，突破类别失衡瓶颈；随着迭代深入，逐步提升 β_t 权重，增大标签平滑的约束比例。通过软化目标概率分布（将硬标签重构为含均匀噪声的软标签），有效抑制模型对训练集噪声的过度记忆，平滑损失曲面并促使优化轨迹逃离尖锐局部极小值。该方案实现了难例挖掘与泛化维稳的自适应迁移，显著校准了模型输出的置信度，为实际部署中的风险阈值划定提供稳定基础。

3.4 研究的整体架构

为验证反馈驱动的动态数据增强策略能够系统化落地，本文构建了层次分明、数据流与反馈信号紧密耦合的整体架构。如图3.2所示，整个研究框架由三大核心方案协同驱动，涵盖数据预处理与特征提纯、动态增强与数据回流、损失计算与梯度优化三个逻辑阶段，形成完整的闭环优化体系。

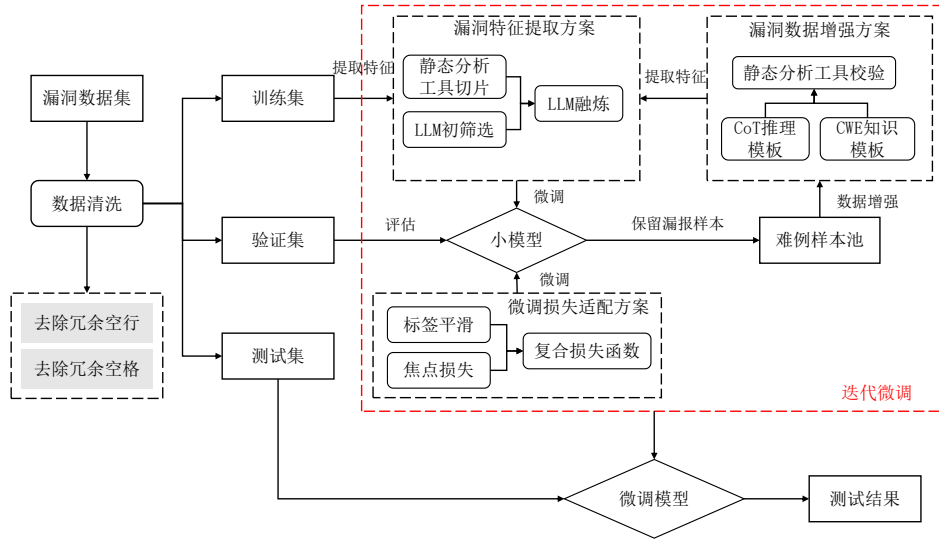


图 3.2 基于大语言模型的漏洞检测增强整体架构

- (1) **漏洞特征提取方案**：原始函数首先经过定向空白字符过滤机制，剔除行首缩进与无逻辑分隔的连续空行，释放无效 Token 占用的序列预算。随后进入漏洞特征提取方案，采用静态 CPG 切片与 LLM 语义初筛双路并行架构。Joern 提供严谨的依赖路径追踪，确立切片语法骨架；LLM 执行跨上下文语义校准与噪声过滤。经融合精炼后，输出长度紧凑、语法密度均衡的标准化代码片段，消除正负样本在序列跨度与控制流复杂度上的系统性偏差。
- (2) **漏洞数据增强方案**：模型完成首轮微调后，验证集输出的 FN 样本作为关键触发信号，驱动漏洞数据增强方案。该方案通过 CoT 推理模板与 CWE 知识库模板并行注入，引导大模型执行结构化代码变异与类型约束生成。候选样本统一汇入 Tree-sitter 语法校验流水线，过滤非法结构片段后，将高信噪比增强样本无缝补充至原始训练集。该层构建了“难例捕获—异构提示生成—语法净化—数据迭代”的闭环，确保增强数据精准填补模型当前的认知盲区。
- (3) **微调损失适配方案**：在每次训练迭代中，微调损失适配方案接收高质量特征基

元进行前向预测，并行计算 Focal Loss 与标签平滑损失。动态权重调整器依据当前训练轮次实时生成调度系数 α_t 与 β_t ，对两路损失进行加权融合。融合后的综合损失经反向传播计算梯度并驱动参数更新，实现优化重心的动态迁移。

三大方案通过“特征提纯—难例定向生成—语法质量管控—动态损失引导”的数据链路实现闭环联动。增强方案定向补充的难例为损失函数提供有效的优化靶点，而动态损失确保模型在吸收新样本时不破坏已习得的泛化表征。该架构使整个策略能够在多轮迭代中持续聚焦脆弱特征，为小型漏洞检测模型提供了一条自适应、可演进的性能优化路径，彻底克服了传统静态扩增方法数据分布错配与计算资源浪费的固有缺陷。

3.5 本章小结

本章围绕小型漏洞检测模型在实际部署中面临的输入序列长度受限、正负样本比例极端失衡及小批量微调易过拟合等核心挑战，系统阐述了基于大语言模型的动态数据增强与损失优化研究方案。首先，明确界定了长序列截断语义丢失、静态增强错配、梯度淹没及输出过度自信等关键工程与算法问题，并归纳出三项核心关键问题；其次，提出了以验证反馈为驱动、以知识约束与逻辑推演为核心的设计思路，确立了从“静态盲目扩增”向“动态精准优化”的范式转变路径，并论证了该框架的方法论优势；随后，详细给出了静态分析与 LLM 协同特征提取、CoT 与 CWE 双模板并行定向生成配合 Tree-sitter 语法校验、以及 Focal Loss 与标签平滑动态加权调度三大核心解决方案；最后，描绘了各方案协同工作的整体架构与闭环数据流转机制，阐明了特征提纯层、动态增强层与损失优化层的数据交互逻辑。该研究方案通过结构化先验提取、显式知识注入与动态优化目标的深度融合，为后续第四章的实验环境配置、基线对比验证及消融分析奠定了坚实的理论基础与算法支撑。

4 基于大语言模型的漏洞检测增强研究方案

4.1 数据清洗

在小型漏洞检测模型的微调过程中，上下文窗口的硬性限制使得输入序列的 Token 分配直接关联到特征学习的有效性。原始源代码数据通常保留了开发环境生成的格式痕迹，其中大量存在于行首的缩进空格与代码块间的连续空行，虽然不参与程序的实际执行逻辑，但在模型的分词阶段却会转化为缺乏语义贡献的无效占位符。对于上下文容量严格受限的轻量级架构而言，这种排版噪声的累积会直接挤占本应用于表征核心代码逻辑的序列长度。若直接投入训练，模型有限的注意力资源将被迫分散至这些格式冗余上，导致危险函数调用、异常边界判断等关键漏洞模式的表征空间受到压缩，进而削弱微调阶段梯度更新的聚焦度。

针对这一约束，本研究在数据预处理环节实施了定向的空白字符过滤机制。该策略聚焦于精准剔除代码行首的冗余空格及无实际逻辑分隔作用的空白行，通过逐行扫描与边界匹配，仅剥离纯排版性质的格式噪声。整个清洗过程严格遵循代码语义无损原则，确保所有控制流分支、变量依赖路径及漏洞触发条件均保持原始结构不变。经过规范化处理后，序列中的无效 Token 占比显著下降，模型得以在固定的最大长度限制内容纳更完整的代码上下文。这一操作本质上是对有限表征资源的重新规划，将小模型宝贵的 Token 配额集中导向高信息密度的漏洞特征区域，为后续的损失计算与参数优化提供了更为紧凑且高效的输入基底。

4.2 漏洞特征提取方案

4.2.1 方案流程设计

针对轻量级漏洞检测模型上下文窗口严格受限、难以直接承载完整函数级代码的现实约束，本研究设计了一种“静态分析定位与大语言模型初筛双路并行—大模型融合精炼”的协同式特征提取流程，详细流程如图4.1所示。

该流程旨在严格控制输入序列长度的同时，精准剥离并保留与漏洞判定高度相关的核心代码上下文，为后续模型微调提供高信噪比的训练样本。具体而言，系统采用双路径并行处理架构对输入的待处理函数进行解析：一路径依托静态程序分析工具进行深度图遍历，基于控制流图与数据流追踪技术精确识别变量污点传播路径、边界条

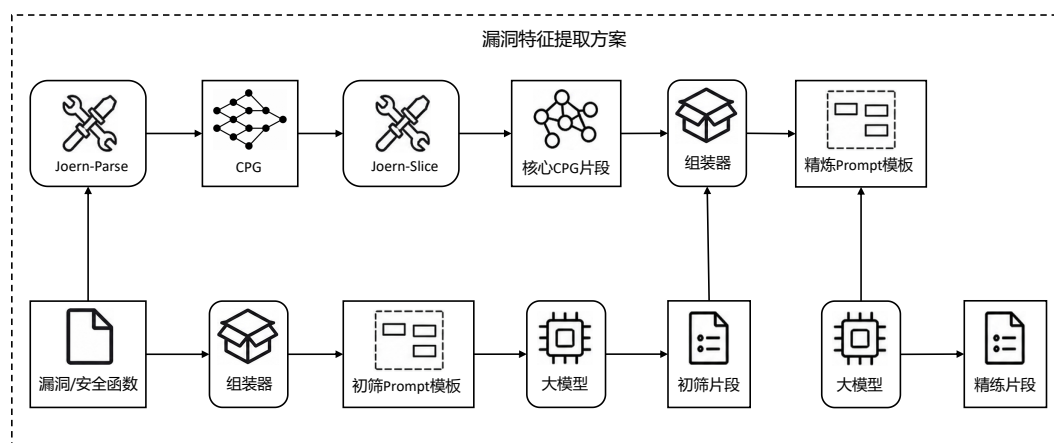


图 4.1 漏洞特征提取方案流程图

件检查缺失及资源释放异常，提取具有严格数据依赖关系的候选代码片段，确保结果具备可验证的执行骨架；另一路径则通过组装器将原始函数结构化封装后送入大语言模型，模型依据组装的结构化提示词从语义逻辑、危险 API 调用模式及异常处理缺失等维度执行初步风险研判，输出初筛片段，以捕捉静态规则难以覆盖的隐蔽型漏洞模式。

在双路输出基础上，系统将静态分析生成的候选切片与初筛片段共同输入大语言模型进行二次语义验证与融合对齐。如图3.2所示，该融合过程以静态工具提取的结构化切片为参考基准，引导大语言模型对初筛结果进行上下文校准与噪声过滤。模型结合静态分析提供的精确依赖路径与自身的深层语义理解能力，智能剔除冗余声明、无关业务逻辑及碎片化代码，自动补全关键上下文信息，最终输出长度紧凑、逻辑闭合且仅保留核心漏洞触发条件的精炼代码片段。该过程有效实现了结构化规则与语义泛化的有机融合，在严格控制序列长度的同时，确保了特征基元的高信噪比与语义完整性。

该协同机制的理论优势在于实现了符号推理与语义表征的深度互补。静态分析路径凭借严谨的数据流建模能力，为融合阶段提供可验证的依赖骨架，有效抑制了大语言模型在长代码理解中的幻觉倾向；而大模型初筛与融合路径则依托强大的跨上下文推理能力，识别业务逻辑缺陷、非常规状态跃迁及隐式类型转换等静态规则难以形式化的隐蔽型漏洞。二者通过融合精炼阶段实现优势互补，显著提升了特征提取的查全率与查准率。此外，为保障小模型微调阶段的数据分布一致性，本流程对漏洞函数与安全函数执行完全对等的提取策略，输出长度分布相近、语法密度均衡的上下文片段，有效消除了正负样本在序列跨度与控制流复杂度上的系统性偏差。综上所述，该特征

提取方案不仅解决了长函数输入受限的技术瓶颈，更通过规则驱动与语义校准的双重约束，为小型漏洞检测模型的高效参数适配奠定了标准化、高信噪比的数据基座。

4.2.2 基于静态分析工具的漏洞特征提取

本研究选择 Joern 作为核心静态分析引擎，其作为开源的跨语言代码分析平台，具备强大的代码属性图（Code Property Graph, CPG）构建能力与可扩展的查询接口。如2.3.1所述，Joern 通过融合抽象语法树（AST）、控制流图（CFG）及程序依赖图（PDG）三种经典程序表征结构，形成统一的 CPG 模型，能够同时刻画代码的语法层次、执行顺序及数据依赖关系，为漏洞特征的精准提取提供了理论基础。

表 4.1 静态分析定义的敏感函数与运算符

类别	名称	风险描述
内存操作函数	malloc	动态内存分配，易引发栈溢出
	free	内存释放，易引发双重释放、悬空指针
	strcpy	无边界检查的字符串复制，易引发缓冲区溢出
	strcat	无边界检查的字符串拼接，易引发缓冲区溢出
	sprintf	无边界检查的格式化输出，易引发缓冲区溢出
	memcpy	内存复制，长度参数未验证易引发溢出
文件操作函数	fopen	文件打开，路径验证不足易引发文件包含漏洞
	read	文件读取，缓冲区大小未验证
系统调用函数	exec	执行系统命令，易引发命令注入
	system	执行 shell 命令，易引发命令注入
敏感运算符	*	未验证指针有效性易引发空指针解引用
	&	地址操作易引发内存泄露
	[]	索引未验证易引发数组越界
	+ - * /	未检查溢出易引发整数溢出

针对常见漏洞模式，本研究基于 Scala 查询语言设计了一套风险识别规则集，聚焦于表4.1所示的敏感函数与运算符。敏感函数涵盖内存操作类、文件操作类及系统调用类等高危 API；敏感运算符则包括指针解引用、地址取址、数组索引及算术运算等易引发缓冲区溢出、整数溢出及空指针解引用的操作符。规则集通过 CPG 的图遍历查询语言实现，能够精确定位这些风险点在 CPG 中的节点位置。详细提取过程如

算法 1 所示。

算法 1 基于 Joern 的漏洞特征提取算法

```

1: 初始化  $V = \emptyset$ 
2: for  $i = 1$  to  $n$  do
3:   步骤 1: CPG 构建
4:    $CPG_i \leftarrow \text{Joern.buildCPG}(f_i)$ 
5:   步骤 2: 风险种子定位
6:    $Seeds \leftarrow \emptyset$ 
7:   for each  $\text{node} \in CPG_i$  do
8:     if  $\text{node.type} \in S_{func}$  or  $\text{node.operator} \in S_{op}$  then
9:        $Seeds \leftarrow Seeds \cup \{\text{node}\}$ 
10:    end if
11:  end for
12:  步骤 3: 双向切片
13:   $SliceNodes \leftarrow \emptyset$ 
14:  for each  $\text{seed} \in Seeds$  do
15:    后向遍历: 追溯定义源头
16:     $BackwardNodes \leftarrow \text{DFS\_Backward}(\text{seed}, CPG_i, D_{max})$ 
17:    前向遍历: 追踪危害传播终点
18:     $ForwardNodes \leftarrow \text{DFS\_Forward}(\text{seed}, CPG_i, D_{max})$ 
19:     $SliceNodes \leftarrow SliceNodes \cup BackwardNodes \cup ForwardNodes$ 
20:  end for
21:  步骤 4: 代码片段还原
22:   $CodeLines \leftarrow \text{extractSourceLines}(SliceNodes)$ 
23:   $CodeLines \leftarrow \text{removeDuplicates}(CodeLines)$ 
24:   $CodeLines \leftarrow \text{sortByLineNumber}(CodeLines)$ 
25:  步骤 5: 输出切片
26:   $v_i \leftarrow \text{reconstructCode}(CodeLines)$ 
27:   $V \leftarrow V \cup \{v_i\}$ 
28: end for

```

首先, 对数据集中的原始函数进行预处理, 将其封装为独立的 C/C++ 源文件并写入临时工作目录, 确保 Joern 能够正确解析编译单元。随后, 调用 Joern 的 CPG 生成器对每个函数进行图结构转换, 生成包含 AST、CFG 及 PDG 信息的完整 CPG。以训练集中 Idx=26 漏洞函数为例, 该函数实现了 RFC1035 的 DNS 标签解析逻辑, 但存在压缩指针偏移量截断、未限制递归深度导致环形引用 DoS、边界检查临界值处理不当等典型内存安全漏洞。其使用 Joern 生成的 CPG 如图4.2所示, 切片前的 CPG 呈现完整的函数结构, 包含所有语法节点、控制流边及数据依赖边, 节点规模达数百个, 直接输入将严重超出小模型的 Token 预算。

在 CPG 构建完成后, 算法执行风险点定位阶段。系统依据预定义的 Scala 规则集, 在 CPG 中搜索匹配敏感函数调用及运算符使用的节点, 标记为”风险种子节点”。随后, 针对每个种子节点启动双向图遍历机制: 后向遍历沿数据依赖边及控制依赖边逆

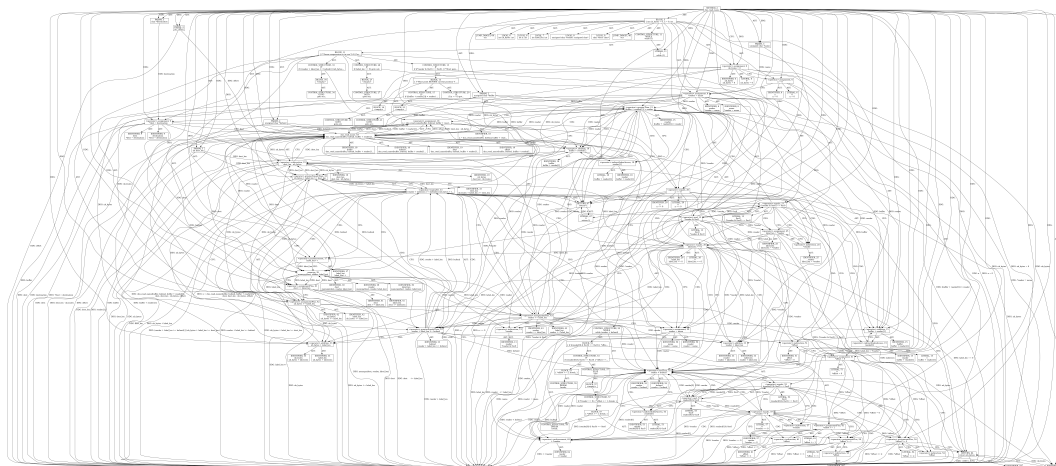


图 4.2 Idx=26 漏洞函数切片前的 CPG 结构图

向追溯，寻找影响风险点的变量定义源头、条件判断分支及外部输入来源；前向遍历则顺向追踪，定位风险点影响的后续操作、返回值传播路径及潜在的危害传播终点。遍历过程采用深度优先搜索（DFS）策略，并设置最大深度阈值以防止图爆炸。

完成双向遍历后，算法收集所有被访问的 CPG 节点及其关联边，构成漏洞相关的子图结构。随后，通过 AST 节点的源码映射关系，将子图中的节点还原为原始代码行号及文本片段。在此过程中，系统执行去重与排序操作，确保还原的代码片段保持原始执行顺序且无冗余行。最终输出的漏洞特征片段仅包含与风险点存在数据流或控制流关联的必要代码上下文，剔除无关的变量声明、初始化及辅助函数调用。如图4.3所示，经过切片处理后的 CPG 规模显著缩减。

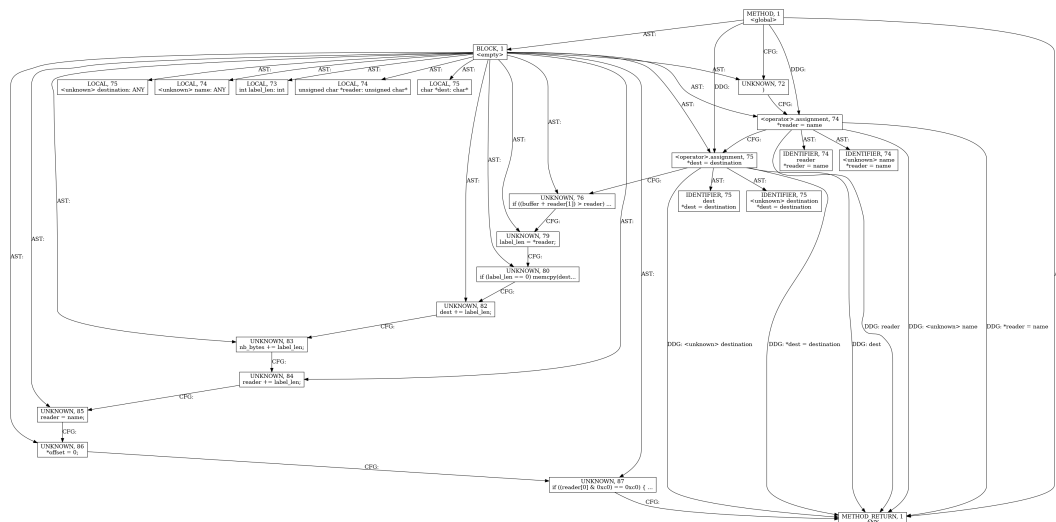


图 4.3 Idx=26 漏洞函数切片后的 CPG 结构图

如图4.4所示，经切片处理后的代码片段在序列规模上得到显著精简。依托 Joern 构建的代码属性图（CPG），系统可精准定位 *reader[1]* 越界访问节点与递归调用控

制边；结合双向切片机制，进一步提取了 $buffer + reader[1]$ 危险指针运算路径及 $dest_len - nb_bytes$ 负值数据传播链路。

```
int dns_read_name(unsigned char*buffer, unsigned char*bufend, unsigned char*name, char*destination, int dest_len, int*offset)
{
    int nb_bytes = 0, n = 0;
    int label_len;
    unsigned char*reader = name;
    char*dest = destination;
    while (1) {
        if ((*reader & 0xc0) == 0xc0) {
            /* Must point BEFORE current position */
            if ((buffer + reader[1]) > reader)
                goto err;
            n = dns_read_name(buffer, bufend, buffer + reader[1], dest, dest_len - nb_bytes, offset);
            if (n == 0)
                goto err;
        }
        label_len = *reader;
        if (label_len == 0)
            goto out;
        if ((reader + label_len >= bufend) || (nb_bytes + label_len >= dest_len))
            goto err;
        label_len++;
        memcpy(dest, reader, label_len);
        dest += label_len;
        nb_bytes += label_len;
        reader += label_len;
    }
out:
    reader = name;
    *offset = 0;
    while (reader < bufend) {
        if ((*reader & 0xc0) == 0xc0) {
            *offset += 2;
            break;
        }
        elseif (*reader == 0) {
            *offset += 1;
            break;
        }
        *offset += 1;
        ++reader;
    }
    return nb_bytes;
err:
    return 0;
}
```

图 4.4 Idx=26 漏洞函数切片前后代码对比（高亮标注关键漏洞模式）

该算法的优势体现在三方面：其一，基于 CPG 的图遍历能够精确捕捉跨语句、跨基本块的长距离依赖关系，克服传统正则匹配无法识别上下文关联的局限；其二，双向切片机制确保提取的片段同时包含漏洞成因（源头）与潜在危害（终点），为小模型提供完整的因果链条；其三，通过规则驱动的自动化提取，保证了大规模数据集处理的一致性与可重复性，为后续模型微调提供了标准化的高质量训练样本。

4.2.3 基于大语言模型的语义初筛与精炼

如图3.2所示，本研究设计的漏洞特征提取方案采用“静态分析定位与大语言模型初筛双路并行一大模型融合精炼”的协同架构。在此架构下，大语言模型提取环节作为流程的汇聚点与精炼核心，承担着多源信息融合与最终特征定型的职能。具体而言，该环节同步接收两路输入：一路是由静态分析工具（Joern）基于 CPG 图遍历提取的“候选片段”，为模型提供严谨的结构化参考；另一路则通过组装器将原始函数填充至预设的提示词模板，送入大语言模型进行初步风险研判，生成保留初步语义风险特征的“初筛片段”，初筛阶段的提示词模板如图4.5所示。

System
You are an expert C/C++ security analyst. Your task is to extract a semantically complete, concise Vulnerability-Relevant Region from the provided function. Strictly follow the extraction rules provided in the user message and output only the final code block.
User
Please analyze the following C/C++ function and Extract the Vulnerability-Relevant Region by strictly adhering to the rules below: [Extraction Rules] 1. Identify the Vulnerability Seed: Locate the exact vulnerable lines. Prioritize modified lines if diff context is implied. 2. Expand via Data Dependency: Include definitions and sources of variables used in the vulnerable line. Trace back to untrusted inputs. 3. Expand via Control Dependency: Include conditional statements that guard or trigger the vulnerable line. If the vulnerability stems from a MISSING check, include the surrounding logic demonstrating the absence of validation. 4. Basic Block Closure: Ensure the snippet contains the complete basic block. Do not truncate mid-logic. 5. Length & Density Control: Target 5–20 lines demonstrating a 'Context + Action' pattern. Avoid single-line extractions without context. Exclude unrelated utility code, comments, or distant functions. [Output Constraints] - Return ONLY the extracted code within a single ```cpp block. - Preserve original indentation, variable names, and line structure. - Ensure the snippet is syntactically self-contained and independently understandable. [Function to Analyze] <pre> '''cpp {code} ''' </pre>

图 4.5 LLM 初筛提示词模板

该提示词模板采用双层结构设计：系统层指令明确界定模型的角色定位与核心任务，要求模型以安全分析专家的视角，从提供的函数中提取语义完整且精简的漏洞相关代码区域，并严格遵循用户层指令中提供的提取规则；用户层指令则详细规定了五条核心提取规则与输出约束。

在提取规则方面，第一条规则要求模型精确定位函数中的漏洞发生位置，识别潜在的危险代码行；第二条规则强调沿数据依赖方向追溯关键变量的定义源头与输入传播路径，确保捕获未信任数据的完整流转链条；第三条规则要求沿控制依赖方向补全

触发漏洞的条件语句，若漏洞源于检查缺失，则保留展示验证逻辑缺失的周围代码；第四条规则确保提取片段包含完整的逻辑基本块，避免在条件分支或循环体的中间位置截断，维持代码的语法闭合性；第五条规则通过长度与密度控制机制，将输出约束在合理范围内，聚焦“上下文 + 动作”模式，剔除无关的辅助代码、注释或远距离函数声明。

在输出约束方面，模板要求模型仅返回封装于单一代码块中的精炼片段，保留原始缩进格式、变量命名及行结构，确保提取结果在语法上自包含且可独立理解。该初筛片段虽已具备初步的语义完整性与漏洞相关性，但仍可能包含冗余上下文或遗漏隐式依赖，需与静态分析生成的候选片段共同输入至后续的大模型融合精炼环节，进行二次语义验证与对齐，最终形成适配小模型输入限制的高质量特征基元。

在此基础上，最终的大语言模型依托精心设计的结构化提示词，对两路输入进行深度对齐与二次语义提炼，执行噪声过滤、边界校验及逻辑缝合，最终输出严格适配小模型 Token 预算的高质量漏洞函数片段或者安全函数片段，精炼阶段的提示词模板如图4.6所示。

System
You are an expert C/C++ security analyst. Your task is to fuse and refine two code snippets into a semantically complete, concise vulnerability-relevant region. Strictly follow the refinement rules and output ONLY the final code block.
User
Refine the following two code snippets into one high-quality vulnerability-relevant region: [Static Slice] ```cpp {static_slice} ``` [Initial Screening] ```cpp {initial_screening} ``` [Refinement Rules] 1. Consistency & Completion: Align both snippets. Ensure all variable definitions, data sources, and control conditions are complete. 2. Noise Filtering: Remove redundant declarations, unrelated code, comments, and debug logs. 3. Boundary Validation: Preserve complete basic blocks (include opening/closing braces). Do not truncate mid-logic. 4. Length Control: Target 5–20 lines with high semantic density. Focus on 'Context + Vulnerable Action' pattern. [Output Constraints] - Return ONLY the refined code in a single ```cpp block. - Preserve original indentation and variable names. - Ensure syntactic completeness.

图 4.6 LLM 精炼提示词模板

该提示词模板同样采用双层结构设计：系统层指令明确界定模型的角色定位与核

心任务，要求模型以安全分析专家的视角，对静态分析生成的“候选片段”与大语言模型生成的“初筛片段”进行深度融合与精炼，输出最终的高质量漏洞相关代码区域；用户层指令则详细规定了四条核心精炼规则、输入数据占位符及输出约束。

在精炼规则方面，第一条规则强调一致性与完整性，要求模型对齐静态切片与初筛片段，利用静态分析的结构化优势补全初筛中可能缺失的变量定义、数据源及控制条件，同时利用初筛的语义优势修复静态切片可能存在的逻辑断裂；第二条规则执行噪声过滤，智能剔除冗余声明、无关辅助代码、注释及调试日志，显著提升代码片段的信噪比；第三条规则进行边界校验，确保提取片段包含完整的逻辑基本块（如保留开闭大括号），避免在条件分支或循环体中间截断，维持语法闭合性；第四条规则实施长度控制，将输出约束在 5 至 20 行范围内，聚焦“上下文 + 漏洞动作”模式，确保高语义密度。

在输出约束方面，模板要求模型仅返回封装于单一代码块中的最终精炼片段，保留原始缩进格式与变量命名，确保结果在语法上自包含且可独立理解。与初筛阶段不同，该环节是整个特征提取流程的汇聚点与终点，通过“静态参考 + 初筛语义”的双重校验与互补融合，有效解决了单一方法存在的过度切片或语义碎片化问题，最终形成严格适配小模型 Token 预算、具备高信噪比与完整因果链条的标准训练样本。

综上所述，漏洞特征提取方案通过“静态分析定位与大语言模型初筛双路并行—大模型融合精炼”的协同机制，实现了对原始函数代码的深度语义解析与精准特征提纯。该方案有效融合了静态分析工具的结构化严谨性与大语言模型的跨上下文推理能力，规避了单一技术路径下的过度切割或语义碎片化风险。最终输出的标准化代码片段在严格契合小模型输入约束的前提下，完整保留了漏洞判定的核心因果链条，为后续检测模型的定向微调与动态数据增强流程提供了高信噪比、强一致性的优质特征基元。

4.3 漏洞数据增强方案

4.3.1 方案流程设计

针对现有数据集中漏洞样本数量过少、正负样本比例严重失衡的问题，本研究设计了一种“CoT 推理模板 + CWE 知识注入模板”并行增强的协同式数据增强流程，详细流程如图4.7所示。

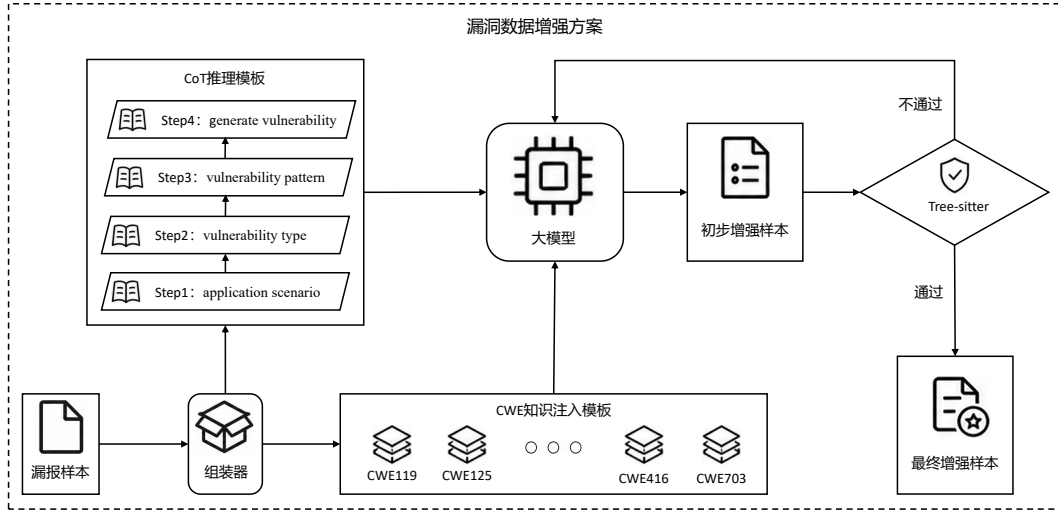


图 4.7 漏洞数据增强方案流程图

该流程旨在以模型在验证集上的检测盲区为核心反馈信号，定向生成高保真、可编译的漏洞增强样本，为后续微调提供针对性的正样本补给。具体而言，系统首先捕获验证阶段的假阴性漏报样本作为增强种子，将其送入大语言模型进行定向变异与注入；在此过程中，方案采用双模板并行注入架构：一路径加载定制化的思维链提示模板，从漏洞成因剖析、类型界定、攻击模式总结到代码样例生成进行结构化推理引导；另一路径则并行注入对应 CWE 类别的专属提示模板，将官方漏洞描述、典型利用路径及修复范式等显式领域知识注入生成过程。二者协同驱动教师模型突破随机增生的局限，实现语义一致、逻辑严密的漏洞代码定向生成。

在生成候选样本的基础上，系统引入 Tree-sitter 语法校验机制对输出结果进行严格的静态结构验证。该环节作为质量管控的核心节点，对候选代码进行词法解析与抽象语法树（AST）重构，精准检测括号匹配缺失、类型声明错误、未定义引用及控制流断裂等语法缺陷。若校验通过，则将符合编译规范的高质量漏洞函数直接回流至训练集；若校验失败，系统则自动将错误上下文与语法约束反馈至教师模型，触发迭代修正循环。该“生成—校验—反馈—再生成”的闭环机制有效规避了传统大模型生成代码中常见的幻觉与语法破碎问题，确保增强样本在具备高语义多样性的同时，满足实际部署环境对代码可执行性的硬性要求。

该协同机制的理论优势在于实现了领域知识引导与语法刚性约束的深度耦合。CoT 与 CWE 双模板的并行注入，使大语言模型摆脱了盲目仿写的生成范式，转而基于严密的逻辑推演与权威的安全规范进行代码重构，显著提升了生成样本的漏洞特征纯度与类型针对性；而 Tree-sitter 的介入则将自然语言生成的柔性过程锚定于编程语

言的形式化语法规则之上，从根本上阻断了无效样本对模型梯度更新的干扰。二者通过闭环反馈链路实现动态调优，确保每一轮增强数据均精准填补模型当前的认知盲区。综上所述，该数据增强方案不仅突破了传统静态扩增方法的盲目性与低质性瓶颈，更通过知识驱动与语法校验的双重保障，为小漏洞检测模型的鲁棒性提升与泛化能力突破奠定了高价值、可迭代的数据基座。

4.3.2 思维链引导的定向增强

针对现有数据增强方法中直接指令生成导致的样本噪声高、漏洞特征漂移及语义连贯性不足等问题，本研究借鉴 GVI 框架^[63]中人类安全专家漏洞挖掘的认知流水线，设计了一种四步递进式思维链提示模板，模板如图4.8所示。

System
You are an expert C/C++ security analyst in vulnerability pattern synthesis. Your objective is to generate high-quality, realistic vulnerable C/C++ functions strictly for academic research and model fine-tuning. You will strictly follow a structured Chain-of-Thought (COT) workflow:
Step 1: Application Scenario Analysis
```cpp {VulFunc} ```  Please analyze the typical application Scenario of the provided C/C++ function.
Step 2: Vulnerability Type Identification
Please identify the specific security vulnerability type found in the provided C/C++ function.
Step 3: Vulnerability Pattern Extraction
Please extract the core vulnerability pattern found in the provided C/C++ function.
Step 4: Generate Similar Vulnerable Functions
Please generate exactly {K} independent, high-quality vulnerable functions that align with the target application scenario and extracted vulnerability pattern.

图 4.8 CoT 提示模板

该模板将复杂的漏洞想象任务解耦为“漏洞原因分析—漏洞类型识别—漏洞模式提取—相似函数生成”四个原子化推理阶段，通过构建隐式信息流引导大语言模型进行结构化、可解释的定向生成。具体而言：

第一步：漏洞原因分析。该阶段旨在确立生成任务的语义上下文边界。系统首先将种子函数输入大语言模型，要求其从应用编程上下文切入，剖析代码的业务逻辑与控制流结构，精准定位触发安全缺陷的根因。此设计模拟了人类专家“先理解程序行

为，后寻找潜在威胁”的审计直觉，为后续推理提供可靠的场景锚点，避免模型在无约束条件下陷入随机想象或脱离实际编程语境。

第二步：漏洞类型识别。在明确漏洞成因的基础上，本阶段侧重于将非结构化的代码缺陷映射至标准化的安全分类体系。模型依托预训练阶段内隐的庞大漏洞知识库，将第一步提取的根因与 CWE 分类标准进行对齐，输出具体的漏洞类别标识及其核心触发机制。引入类型识别的目的在于为生成过程注入显式的领域先验知识，确保后续产出的样本在漏洞属性上具备明确的学术定义与可追溯性，有效规避类型混淆与特征模糊问题，使增强数据能够精准覆盖模型当前的认知盲区。

第三步：漏洞模式提取。该阶段是实现从“具体实例”到“抽象规则”跃迁的关键环节。模型需进一步剥离种子函数的表层语法差异，提炼出跨越代码变体的核心漏洞模式。具体包括：识别关键的数据依赖链、控制流守卫条件、以及“上下文 + 危险操作”的结构化序列。通过模式抽象，系统将多行、跨基本块的复杂漏洞特征固化为准生成模板，从根本上克服了传统单行注入方法无法刻画真实世界多行漏洞的局限性，为高保真样本合成奠定结构基础。

第四步：相似漏洞函数生成。作为 CoT 链路的终端执行节点，本阶段综合前三步沉淀的应用场景约束、CWE 类型标识与抽象漏洞模式，驱动大语言模型执行定向代码合成。模型被指令从零构建指定数量的独立目标函数，严格遵循“语义完整、语法闭合、长度可控”的输出规范。该步骤不仅要求生成代码在逻辑上复现提取的漏洞模式，还需在变量命名、业务上下文及控制流编排上呈现多样化变体，从而实现训练样本在“漏洞本质一致性”与“代码表象多样性”之间的精准平衡。

上述四步 CoT 设计并非简单的任务拆分，而是构建了严密的递进式推理闭环。每一步的输出均作为隐式上下文注入下一阶段，形成累积性的信息增益。以训练集中 Idx=62 的 BZIP2 解压缩漏洞函数为例，COT 引导 LLM 的生成样本如图4.9所示。

第一步首先分析出该函数应用于 BZIP2 解压缩数据处理场景，涉及 Huffman 解码与缓冲区管理；第二步精准识别漏洞类型为缓冲区溢出；第三步进一步抽象出“缺失缓冲区边界检查”的漏洞模式，明确了未校验写入长度即操作内存的根因；第四步综合上述属性，从零生成了 4 个独立变体函数，这些样本在保持“解压缩场景 + 缓冲区溢出”核心特征的同时，在变量命名与控制流结构上呈现多样化。相较于单次提示，该分步策略有效抑制了大语言模型在长程代码生成中的幻觉倾向与逻辑断裂，使增强样本的生成过程从“盲目仿写”转变为“规则驱动的结构化推理”。最终输出的代码

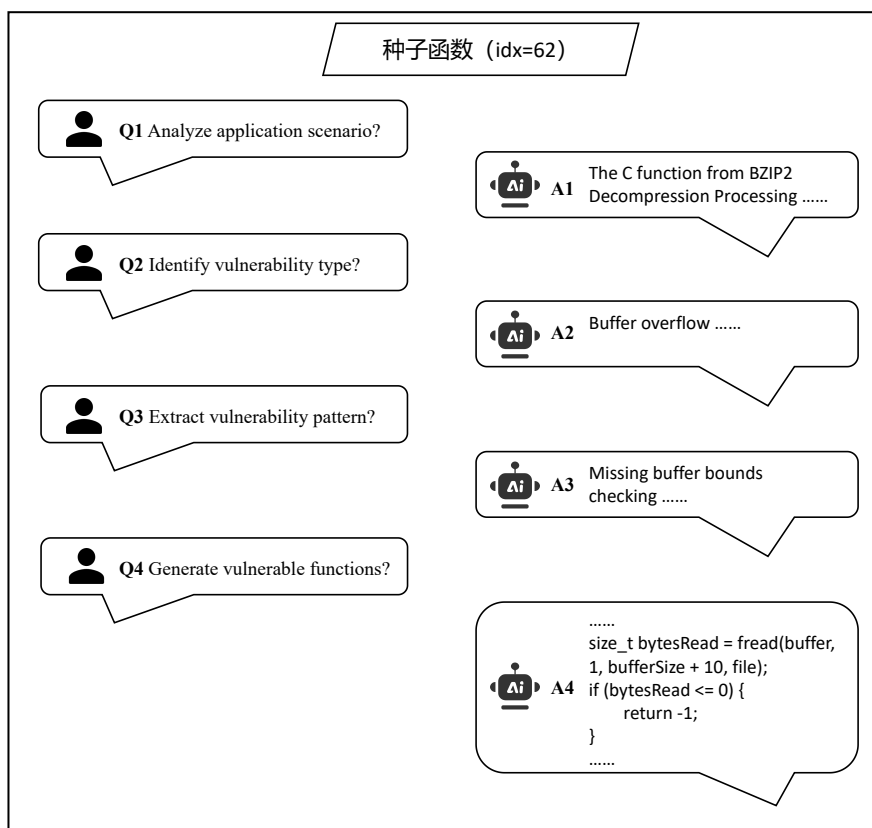


图 4.9 COT 引导 LLM 生成漏洞样本示例

片段不仅具备更高的语法正确率与漏洞真实性，更在特征分布上与原始数据集保持高度同构，为后续小模型的微调提供了高信噪比、强针对性的正样本补给。

### 4.3.3 CWE 知识注入机制

针对现有数据增强方法多局限于对漏洞代码本体的直接变异或注入，而忽视数据集中蕴含的高阶语义元数据，例如漏洞类型的信息，本研究提出了一种基于 CWE 分类体系的知识引导型数据增强策略。该策略旨在将官方标准化的漏洞先验知识显式注入大语言模型生成过程，实现从“无约束盲目生成”向“领域知识驱动定向增强”的范式转变。如第2.1节所述，CWE 是由 MITRE 组织维护的通用弱点枚举标准，为软件安全缺陷提供了系统化的分类框架与权威描述。每个 CWE 条目均对应独立的官方文档页面，结构化地涵盖了漏洞描述、常见后果及典型表现形式等关键要素。这些标准化知识不仅明确了漏洞的本质特征与触发条件，更为大语言模型的定向代码生成提供了可解释、可验证的语义约束。以本研究采用的 PrimeVul 数据集为例，其漏洞类型的 CWE 分布如图4.10所示。

从统计结果可见，数据集中的漏洞样本呈现显著的长尾分布特征。考虑到 Prime-

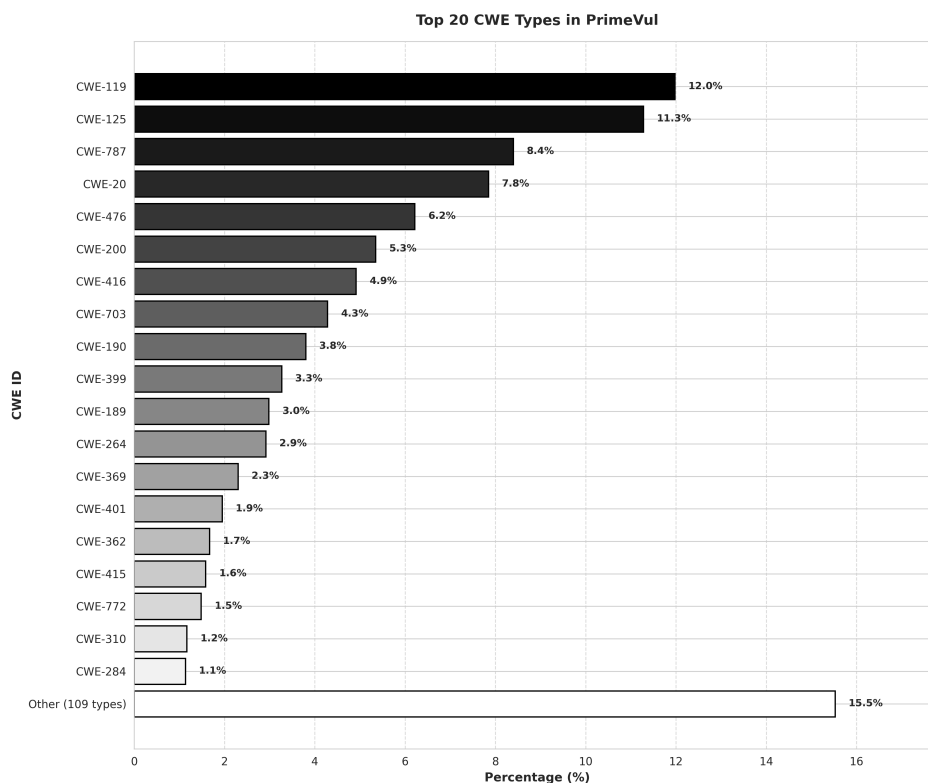


图 4.10 PrimeVul 数据集中 CWE 分布情况

Vul 数据集中涵盖的 CWE 类型繁多，其中大量长尾 CWE 对应的样本数量极少，缺乏统计代表性；同时，部分 CWE 官方文档未完全公开或信息残缺，难以提取标准化的领域知识。为兼顾增强样本的代表性与知识引导的可行性，本研究选取样本占比最高、文档最完备的前 8 个 CWE 类型作为定向增强的目标。这 8 类漏洞均为工业界与学术界公认的高危缺陷，具体如表 4.2 所示。

上述 8 类 CWE 累计覆盖 PrimeVul 数据集中约 60% 的漏洞样本，构成了小检测模型训练的核心正样本基座。针对这些主流 CWE 类型，本研究设计了一套模块化提示词模板，通用结构如图 4.11 所示。

该模板采用“固定指令框架 + 动态知识注入”的双层架构，核心字段包括：种子函数、漏洞定义、漏洞表现形式、生成任务指令及输出格式约束。在实例化过程中，系统自动抓取 CWE 官方页面中的标准化描述与典型案例，然后将漏洞定义直接引用官方表述，表现形式则嵌入典型案例，确保生成过程严格对齐权威安全规范。对于数据集中剩余的长尾 CWE 类别，由于样本稀疏且官方知识缺失，本研究进一步设计了一套通用型 CWE 提示模板，模板结构如图 4.12 所示。

该模板保留了种子代码与任务指令字段，但将具体的漏洞定义与表现形式抽象为



表 4.2 本研究选取的 8 类代表性高危 CWE 漏洞类型及描述

CWE 名称	漏洞名称	漏洞描述
CWE-119	内存缓冲区操作边界限制不当	程序在对缓冲区进行读写时未有效校验边界，易引发栈/堆溢出，导致程序崩溃
CWE-125	越界读取	程序读取了缓冲区边界之外的内存，可能泄露敏感数据或触发段错误
CWE-787	越界写入	程序向缓冲区边界外写入数据，可能破坏关键内存结构或植入恶意代码
CWE-20	输入验证不当	程序未对外部输入进行充分校验即直接使用，易引发注入攻击、路径遍历或逻辑绕过
CWE-476	空指针解引用	程序在指针为 NULL 时执行解引用操作，易引发程序崩溃或拒绝服务
CWE-200	敏感信息暴露	程序意外向未授权方泄露内存内容、错误栈或配置信息，为攻击者提供侦察线索
CWE-416	释放后使用	程序在释放内存后仍引用原指针，攻击者可利用悬垂指针劫持控制流或泄露数据
CWE-703	未正确检查或处理异常条件	程序忽略函数返回值或异常状态，导致在错误路径下执行未定义行为或绕过安全校验

CWE 类型，转而依赖大语言模型预训练阶段内隐的安全知识库进行隐式推理与补全。通过“专用模板精准引导+通用模板隐式覆盖”的双轨设计，本方案有效平衡了增强样本的语义准确性与类型覆盖率。CWE 知识的显式注入不仅约束了生成代码的漏洞本质特征，避免类型漂移与特征稀释，更通过结构化提示显著降低了大模型的幻觉倾向。最终，该机制为后续小检测模型的定向微调提供了高信噪比、强针对性且分布均衡的正样本补给，从根本上缓解了原始数据集的类别不平衡瓶颈。

#### 4.3.4 语法质量管控

在基于大语言模型的漏洞样本增强生成过程中，尽管模型具备较强的代码合成能力，但受限于生成机制的概率性与上下文窗口约束，输出结果仍可能包含括号不匹配、语句截断、标识符非法或基础类型声明缺失等结构性缺陷。若直接将此类含噪样本注入训练集，不仅会干扰小检测模型的梯度更新方向，还可能引发特征学习过程中的分布偏移与局部过拟合。为此，本研究在增强样本回流前引入 Tree-sitter 静态语法校验环节，作为数据质量控制的关键防线。

鉴于 PrimeVul 数据集全部由 C/C++ 语言构成，本研究调用 Tree-sitter 官方提供的

System
You are an expert C/C++ security analyst. Your task is to generate precise, semantically complete vulnerable functions strictly adhering to the provided rules.
User
You are working on a software security task that requires generating semantically equivalent vulnerable C functions.   [Seed Code]  <pre>{seedfunc}</pre>  [Vulnerability Definition]  <pre>{vulDef}</pre>  [Vulnerability Manifestation]  <pre>{vulManifest}</pre>  [Your Task]   Referring to the [Vulnerability Definition] and [Vulnerability Manifestation] above to ensure the correct vulnerability semantics, and using the Seed Code above only as a stylistic and structural reference, generate <b>{k}</b> new C functions.   [Output Format]   Wrap EACH generated function in the following format:  <pre>```c resultCode ```</pre>

图 4.11 CWE 定制化提示词模板

C/C++ 语言解析模块，工具原理详见2.3.2节。需要明确的是，由于 LLM 生成的漏洞样本通常为函数级代码片段，缺乏必要的头文件引入与程序入口，无法直接通过传统编译器进行完整编译验证。因此，本环节仅执行基础语法结构校验，重点检测词法完整性、括号匹配、语句终止符及基础作用域闭合等静态规则，确保代码片段在语法层面合法自治，而不涉及语义逻辑或运行时行为的深度验证。

具体校验流程如下：当 LLM 完成一轮增强样本生成后，系统自动调用 Tree-sitter 解析器对输出代码进行抽象语法树构建。若解析过程中未触发 ERROR 或 MISSING 节点标记，则判定语法校验通过，该样本被标记为有效并补充至初始训练集，随后统一送入4.2节的漏洞特征提取方案进行处理；若解析失败，系统则记录错误类型并触发重试机制，重新提交原始提示词引导 LLM 生成替代样本。为平衡样本质量与生成效率，避免因反复调用大模型导致计算开销激增与增强周期过长，本研究在实验设置中将最大重试次数阈值设定为  $k = 2$ 。当连续两次生成均未通过语法校验时，系统自动舍弃该增强种子，转而处理下一批待增强样本。

通过该语法校验机制，本研究在极低计算成本下有效过滤了明显的结构非法样本，保障了增强数据集的基础代码质量。校验通过的高质量样本将与原始正负样本混合，进入后续的特征对齐与模型微调阶段，为“难例捕获一定向生成—语法净化—数

System
You are an expert C/C++ security analyst. Your task is to generate precise, semantically complete vulnerable functions strictly adhering to the provided rules.
User
You are working on a software security task that requires generating semantically equivalent vulnerable C functions.   [Seed Code]  <pre>{seedfunc}</pre>  [CWE Type]  <pre>{cweType}</pre>  [Your Task]   Referring to the [Vulnerability Definition] and [Vulnerability Manifestation] above to ensure the correct vulnerability semantics, and using the Seed Code above only as a stylistic and structural reference, generate <b>{k}</b> new C functions.   [Output Format]   Wrap EACH generated function in the following format:  <pre>```c resultCode ```</pre>

图 4.12 通用提示词模板

据迭代”的闭环优化体系提供可靠的数据支撑。

## 4.4 微调损失适配方案

### 4.4.1 方案流程设计

针对传统交叉熵损失函数在小模型微调过程中易受多数类样本主导、导致小批量训练下梯度偏置与过拟合加剧的问题，本研究设计了微调损失适配方案，旨在通过复合损失机制与动态调度策略优化模型的梯度更新方向。该方案的整体数据流与调控逻辑如图4.13所示。

具体而言，方案以经特征提取与增强后的高质量漏洞/安全函数片段为输入，驱动小检测模型完成前向传播并输出预测概率分布。随后，系统采用双路并行计算架构：一路执行 Focal Loss 计算以抑制易分样本的梯度干扰，聚焦难分正样本的特征学习；另一路执行标签平滑损失计算以软化目标分布，缓解模型过度自信。为适配训练不同阶段的优化重心，方案内置动态权重调整器，依据当前训练轮次自适应生成调度系数  $\alpha$  与  $\beta$ ，对两路损失进行加权融合。在微调初期，系统赋予  $\alpha$  较高权重，强化模型对脆弱特征空间的挖掘能力；随着迭代深入，逐步提升  $\beta$  权重，增强泛化约束并校准输出置信度。融合后的综合损失经反向传播算法计算梯度并驱动参数更新，形成“前向预测—动态加权—损失融合—梯度反馈”的闭环优化机制。该流程有效实现了从“难

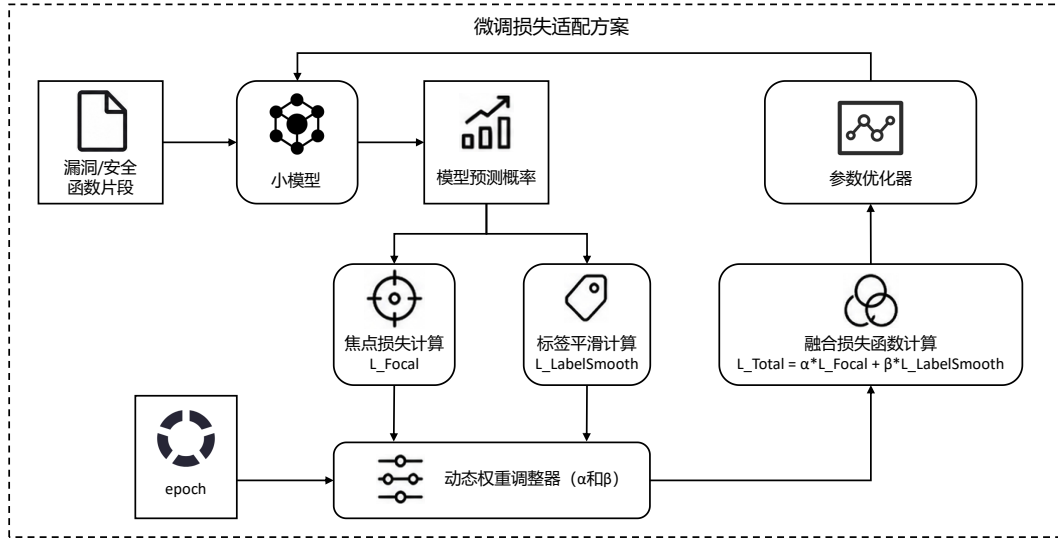


图 4.13 微调损失适配方案流程图

例特征聚焦”到“泛化性能维稳”的平滑过渡，其核心数学表达、参数调度函数及理论分析将在4.4.2至4.4.4节详细展开。

#### 4.4.2 焦点损失（Focal Loss）函数设计

在小漏洞检测模型的微调过程中，训练数据存在极端的类别不平衡现象。以本研究采用的 PrimeVul 数据集为例，安全样本与漏洞样本的比例高达约 33 : 1。传统交叉熵损失函数在处理此类分布时，其梯度更新方向极易被数量庞大的易分负样本（安全代码）所主导。尤其是在显存受限条件下采用小批量训练策略时，单个 Batch 内常出现正样本缺失或比例极低的情况，导致模型梯度严重偏置，倾向于将所有输入预测为负类以最小化整体损失。这种“梯度淹没”效应不仅会削弱模型对稀疏漏洞特征的捕获能力，还会加剧训练初期的震荡与过拟合风险。标准交叉熵损失函数的数学表达式如式4.1所示：

$$L_{CE} = -\frac{1}{N} \sum_{i=1}^N [y_i \log(p_i) + (1 - y_i) \log(1 - p_i)] \quad (4.1)$$

其中， $N$  为批次样本数量， $y_i \in 0, 1$  为真实标签（1 表示漏洞样本，0 表示安全样本）， $p_i$  为模型预测该样本为漏洞的概率。由式4.1可知，交叉熵对所有样本赋予同等权重，无法区分易分样本与难分样本的贡献差异，因而在类别失衡场景下难以引导模型聚焦关键的正类特征。

为突破易分样本对优化过程的垄断，本研究引入 Focal Loss 损失函数。该函数最初由 Lin 等人^[16]提出，旨在解决目标检测中的极端前景——背景不平衡问题，其核心

思想是通过引入可调节的调制因子，动态压缩高置信度样本的损失权重，迫使梯度更新聚焦于难分样本与少数类。其数学表达如式4.2所示：

$$L_{FL}(p_t) = -\alpha_t(1 - p_t)^\gamma \log(p_t) \quad (4.2)$$

其中， $p_t$  为模型对真实类别的预测概率。为保持表述简洁，式4.2以二分类形式展开：当  $\gamma = 1$  时， $p_t = p$ ；当  $\gamma = 0$  时， $p_t = 1 - p$ 。 $\alpha_t$  为类别平衡系数，用于调节正负样本的基础损失权重； $\gamma$  ( $\gamma \geq 0$ ) 为聚焦参数，控制易分样本的降权程度。

调制因子  $(1 - p_t)^\gamma$  的作用机制决定了 Focal Loss 的动态加权特性：当样本被模型正确分类且置信度较高时（即  $p_t \rightarrow 1$ ），调制因子趋近于 0，该样本对总损失的贡献被大幅压缩；反之，当样本难以分类或预测错误时（即  $p_t \rightarrow 0$ ），调制因子趋近于 1，保留其完整的梯度信号。通过调整  $\gamma$  值，可有效重塑损失曲面的形态： $\gamma$  值越大，易分负样本的权重衰减越剧烈，模型对难分正样本的关注度越高。结合  $\alpha_t$  对正样本权重的先验提升，Focal Loss 能够在不增加额外计算开销的前提下，有效缓解小 Batch 训练下的梯度偏置问题，使小模型优先学习漏洞触发的核心模式。然而，单纯依赖 Focal Loss 易导致模型在训练后期过度自信，输出概率分布趋于极端，因此需进一步结合标签平滑策略以校准置信度，具体设计详见4.4.3节。

#### 4.4.3 标签平滑损失函数设计

尽管 Focal Loss 能够有效缓解类别不平衡问题并聚焦难分样本，但在小漏洞检测模型的微调过程中，单纯依赖该损失函数仍可能引发模型过度自信现象。具体而言，当模型在训练后期对某些样本形成强烈判别倾向时，其输出概率分布往往趋近于极端值，导致预测置信度与实际准确率之间产生显著偏差。这种过度拟合训练集噪声的倾向不仅会削弱模型在未见样本上的泛化能力，还会给后续检测阈值的选取带来困难。由于输出概率分布过于尖锐，微小的阈值波动即可能引发召回率与精确率的剧烈震荡，降低部署阶段的稳定性。针对上述问题，本研究引入标签平滑正则化技术，旨在通过软化目标分布约束模型的行为边界，提升其概率校准性与泛化性能。

标签平滑的核心思想在于打破传统交叉熵损失所依赖的硬标签假设。在标准二分类任务中，真实标签通常采用独热编码形式，即正类目标分布为  $[0, 1]$ ，负类为  $[1, 0]$ 。这种确定性标签在训练过程中会驱使模型不断放大正确类别的预测概率，直至逼近理论极值，从而加剧过拟合风险。标签平滑通过引入平滑系数  $\varepsilon$  ( $0 < \varepsilon < 0.5$ )，将硬标

签重构为包含均匀噪声的软标签分布。其数学表达如式4.3所示：

$$L_{LS} = - \sum_{k=1}^K \left[ (1 - \varepsilon) \cdot 1(k = y) + \frac{\varepsilon}{K} \right] \log(p_k) \quad (4.3)$$

其中， $K$  为类别总数（本研究为二分类任务，故  $K = 2$ ）， $1(k = y)$  为指示函数， $p_k$  为模型预测样本属于第  $k$  类的概率。以二分类场景为例，当真实标签为 1 时，平滑后的目标分布变为  $[\varepsilon/2, 1 - \varepsilon/2]$ ；当真实标签为 0 时，则变为  $[1 - \varepsilon/2, \varepsilon/2]$ 。该机制通过向非真实类别分配少量概率质量，有效阻止了模型输出概率向 0 或 1 的无限逼近，迫使网络在优化过程中保持适度的预测不确定性。

从优化动力学角度分析，标签平滑对模型泛化能力的提升主要体现在三个层面。首先，软标签约束显著抑制了模型对训练样本的过度记忆，通过正则化效应平滑了损失曲面，使优化轨迹更易逃离尖锐的局部极小值，从而增强对分布外样本的适应性。其次，该技术有效改善了模型的概率校准性，使得输出置信度能够更真实地反映分类正确的先验概率，这对于安全检测场景中依赖阈值进行风险分级与误报控制具有直接的工程价值。最后，由于预测分布不再呈现极端两极化，检测阈值的微小调整不会引发性能指标的剧烈波动，为实际部署中根据业务需求灵活权衡召回率与精确率提供了稳定的决策基础。本研究参照相关文献的经验建议，将平滑系数设定为  $\varepsilon = 0.1$ ，该取值在防止过拟合与保留类别判别力之间取得了经验性平衡。

然而，标签平滑的均匀噪声注入机制无法区分样本的难易程度，对所有样本施加相同的平滑约束，可能导致难分正样本的梯度信号被进一步稀释。因此，单纯使用标签平滑无法根本解决 PrimeVul 数据集中极端类别不平衡带来的优化困境。为此，本研究在 4.4.4 节提出融合 Focal Loss 与标签平滑的复合损失函数，并设计动态权重调度机制，以实现难例挖掘与泛化维稳的协同优化。

#### 4.4.4 动态加权复合损失函数

尽管 Focal Loss 与标签平滑在各自维度上缓解了类别不平衡与模型过度自信问题，但单一损失函数在复杂漏洞检测任务中仍存在优化目标的局限性。Focal Loss 虽能聚焦难分正样本，但在训练后期易导致模型输出概率分布过度尖锐，削弱泛化边界；而标签平滑虽能软化目标分布，其均匀的噪声注入机制却可能稀释稀缺正样本的梯度信号。为兼顾难例挖掘与置信度校准的双重需求，本研究构建了一种动态加权复合损

失函数，其数学表达如式4.4所示：

$$L_{\text{total}} = \alpha_t \cdot L_{\text{FL}} + \beta_t \cdot L_{\text{LS}} \quad (4.4)$$

其中， $\alpha_t$  与  $\beta_t$  为随训练进程动态演化的权重系数，分别控制 Focal Loss 与标签平滑在综合目标中的贡献比例。

该复合损失的核心创新在于引入基于训练演进的动态权重调度策略，以适配小模型在不同微调阶段的梯度更新需求。在训练初期，模型参数处于随机初始化状态，决策边界尚未形成。受 PrimeVul 数据集极端类别比例影响，小批量采样极易出现全负样本或正样本极度稀缺的情况。若此时均衡施加损失权重，模型梯度将被大量易分负样本主导，陷入局部平庸解甚至引发模型坍塌。为此，调度机制赋予  $\alpha_t$  较高的初始权重，强化 Focal Loss 对正样本及难分样本的聚焦能力，迫使模型优先学习漏洞触发的核心特征模式，有效规避初期全 0 批次导致的梯度偏置与过拟合风险。

随着微调迭代的深入，模型已初步掌握漏洞判别的基本逻辑，此时优化重心需从“特征捕捉”向“边界校准”迁移。调度机制逐步降低  $\alpha_t$  比重，同步提升  $\beta_t$  权重，增大标签平滑在综合损失中的约束比例。这一渐进式转变具有双重理论意义：其一，软标签分布的引入有效抑制了模型对训练集噪声的过度拟合，平滑损失曲面，促使模型逃离尖锐的局部极小值；其二，通过限制输出概率向极端值收敛，提升了预测置信度与真实准确率的一致性（即概率校准性），使模型在未见样本上展现出更稳定的泛化性能。 $\alpha_t$  与  $\beta_t$  的平滑过渡确保了梯度更新方向的连续性与稳定性，避免了损失函数突变引发的训练震荡。

综上所述，式4.4并非两项损失函数的简单线性叠加，而是通过动态权重调度实现了优化目标的时空解耦与协同互补。在微观层面， $\alpha$  与  $\beta$  的实时调节精准响应了小批量训练下的分布波动与模型学习状态；在宏观层面，该机制与4.3节的动态数据增强流程形成闭环联动——增强方案定向补充的高难正样本为 Focal Loss 提供了有效的优化靶点，而标签平滑则确保模型在吸收新样本时不破坏已习得的泛化表征。该复合损失函数作为微调损失适配方案的核心引擎，从根本上重构了小漏洞检测模型的梯度更新范式，为后续高精度阈值划定与稳定部署奠定了可靠的优化基础。

## 4.5 本章小结

本章围绕小漏洞检测模型在实际部署中面临的输入序列长度受限、正负样本比例极端失衡及小批量微调易过拟合等核心挑战，系统阐述了一种基于验证反馈的动态数据增强与损失优化策略。首先，针对原始代码格式噪声挤占 Token 预算的问题，设计了定向空白字符过滤机制，在严格保持代码语义无损的前提下提升输入序列的信息密度。其次，构建了“静态分析定位与大语言模型初筛双路并行——大模型融合精炼”的漏洞特征提取方案，依托 Joern 代码属性图实现精准依赖追溯，结合大语言模型的跨上下文语义理解能力，有效剥离冗余业务逻辑，输出长度适配、因果链条完整的高信噪比代码片段。

针对验证阶段暴露的假阴性难例，本章提出了“CoT 推理模板 +CWE 知识注入模板”并行驱动的定向数据增强机制。通过思维链引导结构化推理与 CWE 官方知识显式注入，突破传统随机增强的盲目性；同时引入 Tree-sitter 静态语法校验与有限重试闭环，从根本上保障生成样本的语法合法性与漏洞特征纯度。在模型微调阶段，设计了融合 Focal Loss 与标签平滑的动态加权复合损失函数，通过基于训练演进的权重调度策略（ $\alpha_t$  与  $\beta_t$  自适应调节），实现了训练初期聚焦难分正样本与训练后期强化概率校准及泛化维稳的平滑过渡，有效缓解了极端类别比例与小批量采样带来的梯度偏置与过拟合风险。

上述方案并非孤立运行，而是通过“特征提纯——难例定向生成——语法质量管控——动态损失引导”的数据流与反馈信号紧密耦合，形成完整的自适应优化闭环。该策略不仅突破了传统静态扩增方法的数据分布错配瓶颈，更通过知识驱动与优化机制的深度融合，为小检测模型的鲁棒性提升奠定了高价值、可迭代的数据与算法基座。下一章将基于上述理论设计与方案实现，详细阐述实验环境配置、数据集选取与评估指标设定，并通过对比实验、消融分析及迭代参数敏感性测试，全面验证本策略的有效性与工程实用价值。